

Chapter - 2

Syntax definition: A syntax definition is a generalization of a content-free grammar in which each grammar symbol has an associated set of attributes, partitioned into two subsets called the inherited attributes of that grammar symbol.

→ A grammar naturally describes the hierarchical structure of most programming language constructs.

□ If (expression) statement else statement

→ using the variable 'expr' to denote an expression and the variable 'stmt' to denote a statement, this structuring rule can be expressed as:

$$\text{stmt} \rightarrow \text{if (expr) stmt else stmt}$$

→ Such a rule is called a production.

→ In a production, lexical elements like the keyword 'if' and the parentheses are called terminals.

→ Variables like 'expr' and 'stmt' represent sequences of terminals (tokens) and are called non-terminals.

Example:

```
for (int; condition; increment) {  
    Statement(s);  
}
```

→ using the variable 'con' to denote a condition, 'int' to denote int, 'inc' to denote increment and 'stmt' to denote statement.

this structuring rule can be expressed as:

```
for (int; con; inc) {  
    stmt(s);  
}
```

⇒ Such a rule is called a production

⇒ In a production lexical elements like keyword 'for' and the parentheses called terminals.

⇒ Variable like 'int', 'con', 'inc', 'stmt' represents sequences of tokens and are called nonterminals.

```
while (expression) {  
    statement(s);  
}
```

```
do while : do {  
    statement(s);  
} while (condition);
```

CFG (Context free grammar): A CFG is a set of rules/production used to generate patterns of strings. It is used to ~~generate~~ describe CFG languages. But they cannot describe all possible language.

A CFG is a 4-tuple $(V/N, \Sigma/T, R/P, s)$ where

⇒ V is a finite set called variables/Non terminals.

⇒ Σ is a finite set called terminals.

⇒ R is a finite set of rules/production.

⇒ s is the start variables.

Example: $9-5+2$

Since a plus or minus sign must appear between two digit, we refer to such expression as;

"lists of digits separated by plus or minus signs"

The following grammar describes the syntax of these expression. The production are

$list \rightarrow list + digit \quad 1$

$list \rightarrow list - digit \quad 2$

$list \rightarrow digit \quad 3$

$digit \rightarrow 0/1/2/3/4/5/6/7/8/9 \quad 4$

The bodies of the three production with non terminals list as head equivalently can be grouped:

$list \rightarrow list + digit \mid list - digit \mid digit$

The terminals of the grammar are the symbol:

$+ - 0 1 2 3 4 5 6 7 8 9$

Left linear Grammar: In a grammar if all productions are in the form $A \rightarrow B\alpha$ or $A \rightarrow \alpha$, where $A, B \in N_N$ and $\alpha \in \Sigma^*$ then the grammar is called left linear grammar.

Ex: $A \rightarrow Aa \mid Ba \mid b$

Right linear Grammar: In a grammar, if all productions are in the form $A \rightarrow \alpha B$ or $A \rightarrow \alpha$ where $A, B \in N_N$ and $\alpha \in \Sigma^*$ then the grammar is called right linear grammar.

Ex: $A \rightarrow aA \mid aB \mid b$

Derivation: A grammar derives strings by beginning with the start symbol and repeatedly replacing a non-terminal by the body of production for that non-terminal.

→ The terminal strings that can be derived from the start symbol form the language defined by the grammar.

→ A derivation tree or parse is an ordered tree that graphically represents the semantic information.

Properties of parse tree:

- Root - Must be labeled by the 1st symbol
- Interior nodes - Labeled by a non-terminal symbol
- Leaves - Labeled by a terminal symbol or ϵ

There are two different approaches to draw a derivation tree -

① Top down approach:

→ Starts with starting symbol S .

→ Goes down to tree leaves using production.

② Bottom up approach:

→ Starts from tree leaves

→ Proceeds upward to the root which is the starting symbol S .

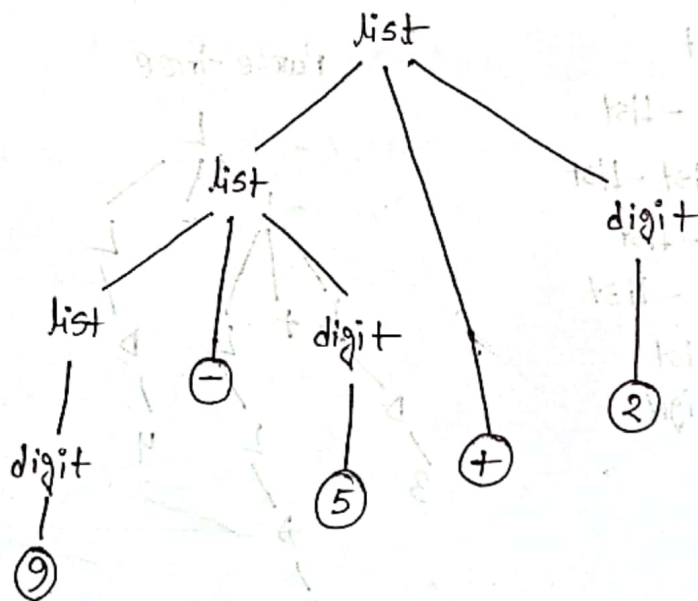
Example-2.2 $9-5+2$

we can deduce that $9-5+2$ is a list as follows:

- a. 9 is a list by production ③, since 9 is a digit.
- b. $9-5$ is a list by production ②, since 9 is a list and 5 is a digit.
- c. $9-5+2$ is a list by production ①, since $9-5$ is a list and 2 is a digit.

The derivation of $9-5+2$

$\text{list} \rightarrow \text{list} + \text{digit} \mid \text{list} - \text{digit} \mid \text{digit}$



Left most derivation: A leftmost derivation is obtained by applying production to the left most variable in each step.

Right most derivation: A rightmost derivation is obtained by applying production to the rightmost variable in each step.

Left most example:

List $\rightarrow$ List + List

List $\rightarrow$ List - List

List $\rightarrow$ digit

digit $\rightarrow$ 0/1/2/3/4/5/6/7/8/9

Input string 3+2-4

List $\rightarrow$ List - List

$\rightarrow$ List + List - List

$\rightarrow$ Digit + List - List

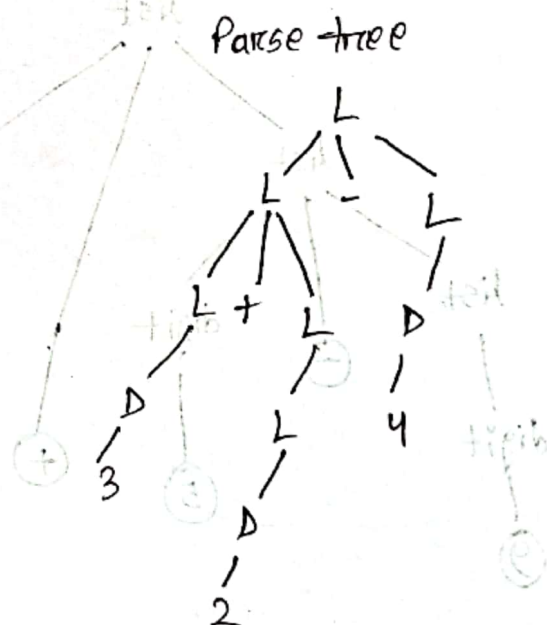
$\rightarrow$ 3 + List - List

$\rightarrow$ 3 + digit - List

$\rightarrow$ 3 + 2 - List

$\rightarrow$ 3 + 2 - digit

$\rightarrow$ 3+2-4



Right most example: 3+2-4

List $\rightarrow$ List + List

$\rightarrow$ List + List - List

$\rightarrow$ List + List - digit

$\rightarrow$ List + List - 4

$\rightarrow$ List + Digit - 4

→ List + 2 - 4

→ Digit + 2 - 4

→ 3 + 2 - 4

Ambiguity in CFG: If a context free grammar G has more than one derivation tree for some string $w \in L(G)$ it is called ambiguous grammar.

A string w is derived ambiguously in context free grammar G if it has two or more different left most derivations. Grammar G is ambiguous if it generates some string ambiguously.

Example: $X \rightarrow X+X \mid X * X \mid X / a$

Derivation 1:

$X \rightarrow X+X$

$\rightarrow a+X$

$\rightarrow a+X * X$

$\rightarrow a+a * X$

$\rightarrow a+a * a$

Derivation 2:

$X \rightarrow X * X$

$\rightarrow X+X * X$

$\rightarrow a+X * X$

$\rightarrow a+a * X$

$\rightarrow a+a * a$

Since, there are two parse trees for a single string

" $a+a * a$ "

The grammar is ambiguous.

- ①
- List $\rightarrow$ List + List
 - List $\rightarrow$ List * List
 - List $\rightarrow$ digit
 - digit $\rightarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

String $\rightarrow$ 1 + 2 * 3

Sol:

- List $\rightarrow$ List + List (Left most)
- $\rightarrow$ digit + List
 - $\rightarrow$ 1 + List
 - $\rightarrow$ 1 + List * List
 - $\rightarrow$ 1 + digit * List
 - $\rightarrow$ 1 + 2 * digit
 - $\rightarrow$ 1 + 2 * 3

- List $\rightarrow$ List * List (Left most)
- $\rightarrow$ List + List * List
 - $\rightarrow$ digit + List * List
 - $\rightarrow$ 1 + digit * List
 - $\rightarrow$ 1 + 2 * digit
 - $\rightarrow$ 1 + 2 * 3

— This grammar is ambiguous

Syntax directed Translation: Syntax directed translation is done by attaching rules or program fragments to production in a grammar. There are two concepts:

1. Attributes: An attribute is any quantity associated with a programming construct.

2. Translation schemes: A translation scheme is a notation for attaching program fragments to the production of a grammar.

⇒ The program fragments are executed when the production is used during syntax analysis.

Postfix Notation: The postfix notation for expression E can be defined inductively as follows:

① If E is a variable or constant, the postfix notation for E is E itself.

② If E is an expression of the form $E_1 \text{ op } E_2$, where op is any binary operation, then the postfix notation for E is $E_1' E_2' \text{ op}$, where E_1' and E_2' are the postfix notations for E_1 and E_2 respectively.

③ If E is an expression of the form (E_1) then the postfix notation for E_1 is also the postfix notation for E .

No parentheses are needed in postfix notation because the position and arity (number of arguments) of the operations permits only one decoding of a postfix expression.

Synthesized attributes: An attribute is said to be synthesized if its value at a parse tree is determined from attribute values at the children of the node.

⇒ synthesized attributes have the desirable property that they can be evaluated during a single bottom up traversal of the parse tree.

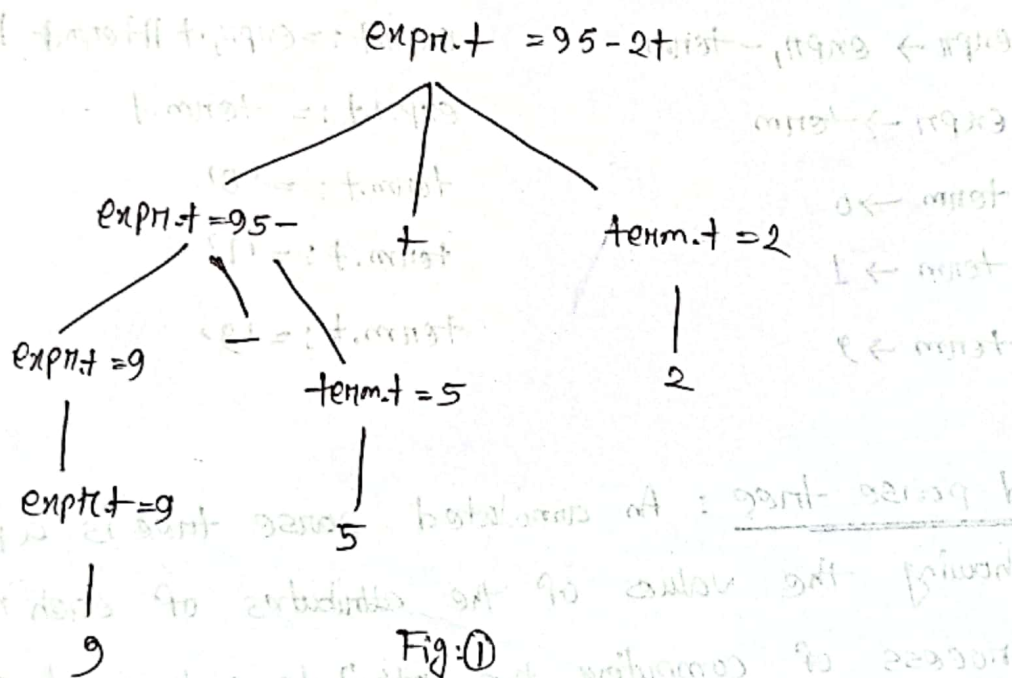


Fig: ①

Semantic rule: The semantic rules connected to the production rule addresses the value of these attributes. The semantic rule defines an attributes as VAL, and an attribute can include any type of data, including strings, numbers, memory locations and complicated records.

Example:

Production	Semantic rule
$expr \rightarrow expr_1 + term$	$expr.t := expr_1.t \parallel term.t \parallel '+'$
$expr \rightarrow expr_1 - term$	$expr.t := expr_1.t \parallel term.t \parallel '-'$
$expr \rightarrow term$	$expr.t := term.t$
$term \rightarrow 0$	$term.t := '0'$
$term \rightarrow 1$	$term.t := '1'$
$term \rightarrow 9$	$term.t := '9'$

Annotated parse tree: An annotated parse tree is a parse tree showing the values of the attributes of each node.

The process of computing the attribute values of the nodes is called annotating or decorating parse tree.

Normal parse tree: Parse tree is the hierarchical representation of terminals or non-terminals.

These symbols represents the derivation of the grammar to yield input strings.